Практическое занятие №1

Тема: Каркас игрового движка и базовая интеграция рендера

Цель: Создать минимальный рабочий каркас игрового движека с поддержкой игрового цикла и визуализации графического вывода через адаптер рендеринга.

Задачи:

1. **Класс Application.** Реализовать класс Application, управляющий жизненным циклом приложения, включая инициализацию, запуск основного цикла и завершение программы.
2. **Система управления состояниями игры.** Разработать структуру для хранения и смены различных игровых режимов (например, меню, игровое состояние, паузы), используя паттерн "Состояния".
3. **Расчёт дельта-времени (deltaTime).** Организовать точную обработку временного интервала между кадрами для обеспечения плавности изменений в игре (анимации, физики и т.п.).
4. **Система логирования.** Настроить механизм ведения журнала для записи важных событий и ошибок.
5. **Адаптер рендеринга (RenderAdapter).** Создать интерфейс адаптера рендеринга, позволяющий подключать различные графические API (OpenGL, DirectX, Vulkan и др.) без внесения изменений в основной код.
6. **Отрисовка тестовых примитивов.** Через созданный адаптер реализовать отображение простейшего геометрического элемента (треугольника или квадрата) в графическом окне.

Глосарий терминов:

Термин	**Определение**
Игровой движок	Набор инструментов и компонентов для разработки компьютерных игр, включающий управление графикой, звуком, физикой и поведением объектов.
Игровой цикл	Основной цикл обновлений логики и рендеринга сцены, повторяемый каждую итерацию.
DeltaTime	Время, прошедшее между двумя последовательными кадрами, используемое для синхронизации анимации и симуляции физики.
RenderAdapter	Паттерн проектирования, обеспечивающий подключение сторонних графических библиотек без привязки к конкретному API.
Состояние игры	Отдельная фаза функционирования игрового движка (игровое меню, игровая сцена, экран загрузки).

Необходимые инструменты:

- **Среда разработки**: Visual Studio 2022, CLion, VS Code.

- **Язык программирования**: C++17 или выше.
- **Графическая библиотека**: OpenGL, DirectX, Vulkan либо готовые фреймворки типа SFML, Raylib.
- **Система сборки**: рекомендуется CMake.
- **Контроль версий**: Git.

Теоретическая справка:

- **Игровой цикл**: Основная логика каждой игры строится вокруг двух фаз: обновление состояния мира (логика) и отрисовка текущего кадра (рендеринг).
- **DeltaTime**: обеспечивает равномерность скорости движений независимо от частоты кадров, важно для точности физических расчетов и анимации.
- **Паттерн Adapter ("Адаптер")**: позволяет абстрагироваться от конкретной реализации графического API, предоставляя единый интерфейс взаимодействия.
- **Система состояний**: упрощает разработку, позволяя динамически менять поведение игры (например, переход из режима загрузки в игру или меню).

Исходные данные:

Владеть основными знаниями C++, понимать принципы ООП и иметь представление о работе с компьютерной графикой.

Порядок выполнения задания:

Шаг 1. Класс Application

- Реализовать базовые методы: инициализация, запуск главного цикла и закрытие приложения.
- Результат: шаблон приложения с простым циклом без функциональности.

Шаг 2. Игровой цикл

- Реализовать переменный или фиксированный шаг обновления.
- Использовать высокоточный таймер для точного вычисления deltatime.
- Результаты: вывод значений deltatime в журнал.

Шаг 3. Система состояний игры

- Создать классы состояний игры (загрузка, основное меню, геймплей) и систему их обработки.
- Возможностью переключаться между разными состояниями.

Шаг 4. Адаптер рендеринга

- Определить интерфейс рендерера с необходимыми методами (инициализация, рисование, освобождение ресурсов).
- Написать реализацию конкретного адаптера для выбранной библиотеки.

Способы подключения собственной системы рендеринга:

1. Прямая интеграция с низкоуровневыми API:

Используйте нативные библиотеки для графики (OpenGL, DirectX, Vulkan) напрямую. Подходит для глубокого изучения устройства графики и низкого уровня оборудования.

2. Использование высокоуровневых библиотек:

Альтернативой является применение универсальных графических фреймворков (SFML, Raylib, SDL). Они облегчают начальное освоение и ускоряют создание прототипов.

3. Гибридный подход:

Сделайте вашу систему рендеринга гибко настраиваемой, чтобы она могла поддерживать как собственные API, так и внешние библиотеки. Это позволит свободно переключаться между технологиями без больших усилий.

Шаг 5. Тестовый примитив

- Используя разработанный адаптер, нарисовать простой геометрический элемент (треугольник или квадрат) в окошке.
- Итог: окно с минимально функционирующим движком и выводимым объектом.

Задания для самостоятельной работы:

- Добавьте возможность поддержки нескольких окон одновременно.
- Реализуйте изменение цвета фона окна через внешний конфиг (.json).
- Если используете OpenGL или Vulkan, добавьте возможность горячего обновления шейдеров.

Рекомендации по выполнению:

- Используйте std::chrono для измерения времени между кадрами.
- Логируйте важные события и возможные ошибки.
- Соблюдайте SOLID-принципы проектирования (особенно Single Responsibility Principle).
- Проверяйте работоспособность каждого модуля до начала интеграции.

Последовательность выполнения:

1. Настройте проект и среду разработки.
2. Реализуйте приложение и главный цикл.
3. Разработайте систему игровых состояний.
4. Подготовьте адаптер рендеринга.

5. Проверьте отрисовку тестового примитива.
6. Выполните тестирование и исправьте найденные проблемы.

Форма отчета:

1. Титульный лист с ФИО студента, группой и названием работы.
2. Подробное описание общей архитектуры вашего решения.
3. Листинги наиболее значимых частей кода.
4. Скриншоты рабочего приложения.
5. Выводы и комментарии по трудностям, возникающим при выполнении задания.

Приложения к отчету:

- Полный исходный код проекта (репозиторий GitHub/GitLab).
- Конфигурационные файлы (CMakeLists.txt, JSON-конфиги и др.).
- Демонстрационное видео работы (необязательно).

Контрольные вопросы:

1. Чем отличается фиксированный и переменный шаги игрового цикла?
2. Зачем необходим RenderAdapter и можно ли обходиться без него?
3. Какова роль deltaTime в управлении физическими процессами и анимациями?
4. Какие преимущества дает использование системы состояний в играх?
5. Как технически организовать смену графического API («горячую загрузку»)?

Рекомендуемая литература:

1. *Game Engine Architecture* — Jason Gregory
2. *«Паттерны проектирования в игровой разработке»* — Роберт Нистром
3. *C++ for Professionals* — Никола Джоссеттис
4. Документации по OpenGL, DirectX, Vulkan
5. Статьи по организации игровых циклов и Entity Component System (ECS) на ресурсах Habr, GameDev.Ru

Примечание:

Это фундаментальная работа, формирующая основу будущих проектов. Особое внимание уделите архитектурной чистоте и удобству расширения вашей реализации.

Дополнительное задание для дальнейшей работы над созданием игрового движка:

Добавьте в свою работу **интеграцию ввода с клавиатуры и мыши**, позволяющую управлять положением тестового объекта на экране. Для этого потребуется расширить существующий класс Application и добавить обработчики событий ввода.

Детали дополнительного задания:

Требования:

1. Реализовать обработчик нажатия клавиш клавиатуры, позволяющий перемещать тестовый примитив (треугольник или квадрат):
    - стрелки вверх/вниз → вертикальное смещение,
    - стрелки вправо/влево → горизонтальное смещение.
2. Обработчик кликов мышью:
    - Левая кнопка мыши → масштабирует объект относительно центра экрана.
    - Правая кнопка мыши → вращает объект вокруг своей оси.
3. Позиция и размеры объекта должны плавно меняться с учётом текущих значений deltaTime.
4. Протестировать функциональность и убедиться, что движение гладкое и соответствует ожиданиям.

Важные рекомендации:

- Используйте событие Input Events из среды разработки (Visual Studio, GLFW, SDL и т.д.) для отслеживания действий игрока.
- Применяйте компонентный подход, выделив отдельный класс или набор функций для обработки вводимых данных.
- Обязательно учитывайте значение deltaTime при изменении позиции и размеров объекта, чтобы обеспечить одинаковую скорость движения вне зависимости от FPS.